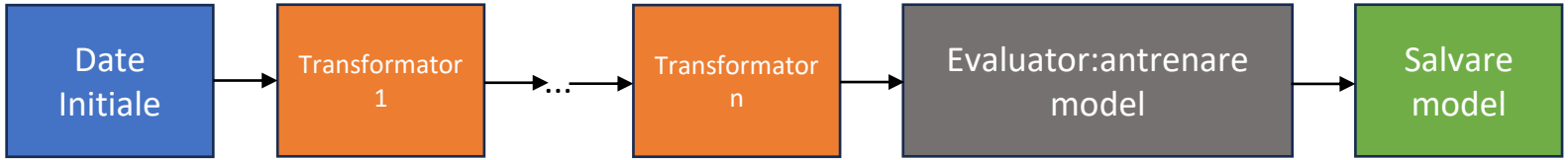
Linii de procesare a datelor

- În general paradigma învățării automate constă în manipularea datelor prin înlănțuirea unor pași
- Pașii se pot împărți în:
  - Pași de transformare a datelor – setul de date de la intrare este prelucrat și la ieșire se obține un alt set de date
    - Exemple: filtrare, eliminare date lipsă, eliminare anomalii, imputare, codificarea datelor, standardizare, discretizare, ingineria atributelor
    - Acești pași sunt executați de transformatoare de date – module dedicate care execută pașii necesari
  - Pasul de antrenare/evaluare a unui model – setul de date de la intrare este folosit pentru a antrena un model de învățare automată, iar la ieșire avem un model care poate fi salvat și reutilizat pe date noi
    - Este un singur astfel de pas, ultimul de pe linia de procesare

Linii de procesare a datelor

- Linii de asamblare:



- Resurse Python:

sklearn.pipeline.Pipeline

class `sklearn.pipeline.Pipeline`(*steps*, *, *memory=None*, *verbose=False*) [source]

Pipeline of transforms with a final estimator.

Sequentially apply a list of transforms and a final estimator. Intermediate steps of the pipeline must be 'transforms', that is, they must implement `fit` and `transform` methods. The final estimator only needs to implement `fit`. The transformers in the pipeline can be cached using `memory` argument.

The purpose of the pipeline is to assemble several steps that can be cross-validated together while setting different parameters. For this, it enables setting parameters of the various steps using their names and the parameter name separated by a `'__'`, as in the example below. A step's estimator may be replaced entirely by setting the parameter with its name to another estimator, or a transformer removed by setting it to `'passthrough'` or `None`.

De ce pipeline: validare încrucișată fără scurgeri

- În validarea încrucișată, datele se împart de mai multe ori (folduri)
- Dacă scalați sau codificați înainte de validare, fiecare fold „vede” statistici din datele lui de test
- Un Pipeline leagă transformările și modelul într-un singur obiect: make_pipeline(pregatire, model)
- cross_val_score reînvață transformările din nou pe fiecare fold, doar pe partea de antrenament
- Același obiect se salvează și se folosește pe date noi, cu aceiași pași